

1. Find 3 API about a domain you care about (Artificial Intelligence)

- OpenAI API
 - Type: REST
 - Base URL: <http://api.openai.com/v1>
 - Authentication method: HTTP Bearer
 - Versioning: Yes, currently /v1
 - Resource Identifier: String / opaque ID
 - Reference: <https://developers.openai.com/api/reference/overview>
- Google Gemini API
 - Type: REST
 - Base URL: <http://generativelanguage.googleapis.com>
 - Authentication method: x-goog-api-key
 - Versioning: Yes, /v1 for stable and /v1beta for beta
 - Resource identifier: String / opaque ID
 - Reference: <https://ai.google.dev/api>
- Anthropic API
 - Type: REST
 - Base URL: <http://api.anthropic.com>
 - Authentication method: Bearer
 - Versioning: Yes
 - Resource identifier: String / opaque ID
 - Reference: <https://platform.claude.com/docs/en/api/overview>

2. Compare 5 core properties of a modern API with what Geewax and Higginbotham define to have a good API (limited in Chapter 1).

In the slide, there are five core properties of a modern API which are contract-first, abstraction, reusability, language-agnostic, and versioned&observable. In API Design Pattern, Geewax recommends 4 characteristics that make an API good including: operational, expressive, simple, and predictable. And in Principles of Web APIs Design, Higginbotham referred to three things including business capabilities, product thinking, and developer experience as the three important elements that make an API effective. Three interpretations sound different but they actually express the same answer to the question “What makes an API good?”. Actually, in PWAD, it is more about design thinking while others are more about

technical thinking; and in Chapter 1 of these two books, 2 properties: language-agnostic and versioned&observable are not clearly recommended.

3. Extended problem 6

```
@app.route("/books", methods=["GET"])
def list_books():
    bid = request.args.get("id")
    if bid is not None:
        try:
            bid = int(bid)
        except (TypeError, ValueError):
            return {"error": "id must be an integer"}, 400
    book = find(bid)
    if not book:
        return {"error": "not found"}, 404
    return jsonify(book), 200

    sort_by = request.args.get("sort")
    if sort_by is not None:
        if sort_by not in ("id", "title", "author", 'year'):
            return {"error": "invalid field"}, 400
        BOOKS.sort(key=lambda b: b[sort_by])
        return jsonify(BOOKS), 200

    n = int(request.args.get("limit", 100))
    BOOKS.sort(key=lambda b: b["id"])
    return jsonify(BOOKS[:n]), 200

# CREATE - POST /books
@app.route("/books", methods=["POST"])
def create_book():
    global _next
    body = request.get_json(silent=True) or {}
    t, a, y = body.get("title"), body.get("author"), body.get("year")
    if not t or not a:
        return {"error": "need title+author"}, 400

    if isinstance(y, str):
        try:
            y = int(y)
```

```
except ValueError:
    return {"error": "year must be an integer"}, 400

if y is not None and (y < 1900):
    return {"error": "year must be larger or equal to 1900"}, 400

book = {"id": _next, "title": t, "author": a, "year": y}
_next += 1; BOOKS.append(book)
return jsonify(book), 201, {"Location": f"/books/{book['id']}"}"
```